

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Relational Database Design & SQL Reference

Name: Prasanna V

Roll No: TU-6253210211041

1. Overview

This document defines a relational database design for managing Products and Categories. It covers table structures, primary/foreign key relationships, sample data operations (insert, update, delete), and a category-wise sales/stock report query.

2. Entity Relationship Overview

A Category can contain many Products, but each Product belongs to exactly one Category. This is a classic one-to-many relationship, implemented using a foreign key (category_id) in the Product table that references the primary key of the Category table.

Category (1) → Product (Many)

3. Table Design

Category Table

category_id: INT, PRIMARY KEY, AUTO_INCREMENT
category_name: VARCHAR(100), NOT NULL, UNIQUE
description: VARCHAR(255), NULL
created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

Product Table

product_id: INT, PRIMARY KEY, AUTO_INCREMENT
product_name: VARCHAR(150), NOT NULL
category_id: INT, FOREIGN KEY → Category(category_id)
price: DECIMAL(10,2), NOT NULL, CHECK (price >= 0)
stock_quantity: INT, NOT NULL, DEFAULT 0
created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

4. SQL: Table Creation & Key Relationships

```
CREATE TABLE Category (  
    category_id INT AUTO_INCREMENT PRIMARY KEY,  
    category_name VARCHAR(100) NOT NULL UNIQUE,  
    description VARCHAR(255),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE Product (  
    product_id INT AUTO_INCREMENT PRIMARY KEY,  
    product_name VARCHAR(150) NOT NULL,  
    category_id INT NOT NULL,  
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
    stock_quantity INT NOT NULL DEFAULT 0,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_product_category  
        FOREIGN KEY (category_id)  
        REFERENCES Category(category_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

ON DELETE RESTRICT prevents deleting a category while products still reference it.

5. Product Insertion, Updating and Deletion

5.1 Insert Categories

```
INSERT INTO Category (category_name, description) VALUES
('Electronics', 'Electronic gadgets and accessories'),
('Groceries', 'Daily food and household essentials'),
('Furniture', 'Home and office furniture');
```

5.2 Insert Products

```
INSERT INTO Product (product_name, category_id, price, stock_quantity) VALUES
('Wireless Mouse', 1, 799.00, 50),
('Bluetooth Speaker', 1, 1999.00, 30),
('Basmati Rice 5kg', 2, 650.00, 100),
('Office Chair', 3, 4500.00, 15);
```

5.3 Update a Product

```
UPDATE Product
SET price = 849.00,
    stock_quantity = stock_quantity + 25
WHERE product_id = 1;
```

5.4 Delete a Product

```
DELETE FROM Product
WHERE product_id = 2;
```

6. Category-wise Product Report

```
SELECT
    c.category_name,
    COUNT(p.product_id) AS total_products,
    SUM(p.stock_quantity) AS total_stock,
    ROUND(SUM(p.price * p.stock_quantity), 2) AS inventory_value
FROM Category c
LEFT JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name
ORDER BY inventory_value DESC;
```

Sample Report:

Electronics — 2 products — 80 stock — ■99,950.00

Furniture — 1 product — 15 stock — ■67,500.00

Groceries — 1 product — 100 stock — ■65,000.00

6.1 Product Listing Within Each Category

```
SELECT
    c.category_name,
    p.product_name,
    p.price,
    p.stock_quantity
FROM Category c
JOIN Product p ON c.category_id = p.category_id
ORDER BY c.category_name, p.product_name;
```

7. Summary

- Category (1) – Product (Many) relationship is enforced through a foreign key.
- Primary keys auto-generate unique IDs.
- The foreign key maintains referential integrity.
- INSERT, UPDATE, and DELETE operations manage records.
- Category-wise reports help analyze stock and inventory value.